

Representacion Tabular de Arboles Binarios

Ronquillo Nunez Braulio
Analisis y Diseno de Algoritmos

Resumen—Los arboles son estructuras jerarquicas fundamentales en algoritmos y estructuras de datos. Un arbol binario restringe cada nodo a tener a lo mas dos hijos, lo que permite definir recorridos recursivos y representaciones compactas. Este reporte describe el modelo teorico de arbol, su representacion mediante tabla de nodos, los recorridos clasicos y la complejidad del programa desarrollado.

Index Terms—arbol binario, recorrido, preorder, inorder, postorder, estructura de datos.

I. INTRODUCCION

Un arbol permite representar informacion con relaciones padre-hijo: sistemas de archivos, expresiones aritmeticas, decisiones, jerarquias y busquedas. A diferencia de una lista, la estructura no es lineal. NIST define un arbol como una estructura que se accede desde una raiz, donde cada nodo puede ser hoja o nodo interno con hijos [1]. Tambien puede verse como un grafo conexo, no dirigido y aciclico.

II. MARCO TEORICO

En un arbol binario, cada nodo tiene como maximo dos hijos: izquierdo y derecho. La distincion entre ambos importa porque el orden de los hijos afecta los recorridos y, en otros contextos, la busqueda. Una definicion recursiva simple establece que un arbol es vacio o esta formado por una raiz y dos subarboles.

Los recorridos principales son:

- *Preorder*: visita raiz, subarbol izquierdo y subarbol derecho.
- *Inorder*: visita subarbol izquierdo, raiz y subarbol derecho.
- *Postorder*: visita subarbol izquierdo, subarbol derecho y raiz.

Estos recorridos visitan cada nodo exactamente una vez. La pagina de notas de MIT OCW para algoritmos incluye arboles binarios como tema de estructuras de datos y recorridos [2].

III. REPRESENTACION TABULAR

El programa usa una tabla con cuatro campos:

Cuadro I
CAMPOS DE CADA NODO

Campo	Significado
id	identificador unico del nodo
valor	dato almacenado
izq	identificador del hijo izquierdo
der	identificador del hijo derecho

El valor -1 indica ausencia de hijo. Con esta informacion se construye un diccionario que permite consultar cualquier nodo

por identificador. Despues, desde la raiz indicada, se ejecutan los recorridos.

IV. VALIDACION

La representacion debe cumplir condiciones minimas:

1. todos los identificadores referenciados deben existir,
2. no debe aparecer un ciclo,
3. un nodo no debe quedar conectado como hijo de varios padres,
4. la raiz debe permitir alcanzar los nodos del arbol.

Estas reglas evitan que una tabla arbitraria se interprete como arbol cuando en realidad describe un grafo invalido.

V. COMPLEJIDAD

Construir la tabla cuesta $O(n)$ para n nodos. Cada recorrido tambien cuesta $O(n)$ porque visita todos los nodos una sola vez. La memoria adicional se debe al diccionario de nodos y a la profundidad de recursion. En un arbol desbalanceado, la profundidad puede ser $O(n)$; en uno balanceado, $O(\log n)$.

VI. CONCLUSION

La representacion tabular es util para observar explicitamente los enlaces entre nodos. Aunque no usa punteros reales, conserva la idea de raiz e hijos. Los recorridos preorder, inorder y postorder muestran como una definicion recursiva se transforma en algoritmos concretos.

REFERENCIAS

- [1] P. E. Black and Algorithms and Theory of Computation Handbook, "tree," Dictionary of Algorithms and Data Structures, NIST, 2017. [Online]. Available: <https://www.nist.gov/dads/HTML/tree.html>
- [2] E. Demaine, J. Ku, and J. Solomon, "6.006 Introduction to Algorithms: Lecture Notes," MIT OpenCourseWare, 2020. [Online]. Available: <https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/pages/lecture-notes/>
- [3] P. Morin, "Open Data Structures," Athabasca University Press, 2013. [Online]. Available: <https://opendatastructures.org/>